

FedCAST: Budget-Aware Federated Stream Clustering over Apache Kafka under Non-IID Data and Network Constraints

Author One^a, Author Two^a, Author Three^a

^aDepartment of Computer Science, University Name, City, Country

Abstract

Edge devices produce unlabeled, non-IID data streams over constrained and heterogeneous links, yet operators need a continuously updated global clustering without collecting raw data. We present FedCAST, a budget-aware federated stream clustering system built on Apache Kafka. Each node summarises its stream with time-decayed cluster features and decides, at every tick, whether an update is worth its network cost and which micro-clusters to send. The decision uses an objective-linked staleness score that provably bounds the coordinator's k-means cost error and centroid displacement. A knapsack-style prioritisation, a scale-free exponentiated dual-ascent threshold, a token bucket with a hard budget guarantee, a link-price term, and a novelty override for clusters seen at a single node complete the policy. The coordinator aggregates staleness-decayed summaries with warm-started weighted k-means and swap local search, which protects node-exclusive clusters. Kafka's per-key ordering, idempotent producers and log-compacted changelog provide exactly-once state and crash recovery without resends, and the complete system runs in under 1 GB of RAM. Across five datasets, seven baselines, non-IID partitions, drift, heterogeneous links and more than 2,000 seeded runs, FedCAST reaches within 5 % of a centralised learner's clustering cost with 10–61 % fewer bytes than the best baseline and up to 8.5× fewer than periodic synchronisation. It ranks first at matched budgets (Friedman $p=2\times 10^{-4}$) and detects emerging clusters 2.5× faster than periodic updates while sending less. On a real broker it recovers from a coordinator crash in about one second with state identical to an uninterrupted run and loses no update during a broker outage.

Keywords: federated clustering, data streams, communication efficiency, non-IID data, Apache Kafka, edge computing

1. Introduction

Network probes, industrial gateways, smart meters and vehicles continuously produce unlabeled data streams at the edge. Operators want a single, continuously updated *global* view of the structure in these streams, such as the traffic profiles present in a network right now or the operating regimes of a machine fleet. Three obstacles stand in the way. First, raw data often cannot leave the device, for regulatory, privacy or volume reasons. Second, the data are *non-IID*: each device sees a different slice of the global distribution, and some patterns are visible at a single device only. Third, links are constrained and heterogeneous: metered cellular uplinks, lossy wireless and intermittent connectivity make every transmitted byte costly.

Stream clustering answers the first obstacle with compact micro-cluster summaries [1, 2]. Distributed variants ship these summaries to a coordinator [3–5], and federated clustering handles heterogeneity on static data [6, 7]. What is missing is a principled answer to the question every edge node faces at every instant: *is sending an update now worth its network cost, and if so, which part of the model should be sent?* Existing systems answer with a fixed period or a data-unit threshold. Periodic updates waste bytes when nothing changes and leave the server stale exactly when the data drift. Thresholds must be re-tuned for every dataset, and neither is connected to the quality of the global clustering or to a byte budget.

This paper presents **FedCAST**, a federated stream clustering

system in which every byte is spent where it reduces the global clustering error most, within a hard per-node budget, on top of Apache Kafka. Our contributions are:

1. **Objective-linked staleness.** A per-micro-cluster score computed on the device from CF vectors and the broadcast global model. Its sum provably bounds the server's k-means cost error (Proposition 1) and the displacement of the global centres (Proposition 2).
2. **A budgeted, prioritised, link-aware send policy.** Knapsack-style selection of the most valuable micro-clusters, a scale-free exponentiated dual-ascent threshold, a token bucket with a hard guarantee and $O(1/n)$ long-run convergence (Proposition 3), a link-price term, and a novelty override for clusters that appear at a single node.
3. **Heterogeneity-aware aggregation.** Staleness-decayed, optionally node-balanced weighted k-means over CFs with swap local search, which keeps node-exclusive clusters from being absorbed by warm-started centres.
4. **A Kafka-native system.** Per-key ordering, idempotent producers with sequence numbers, and a compacted changelog give exactly-once state and crash recovery without any resend from the nodes. The complete system runs in under 1 GB of RAM.
5. **A rigorous evaluation** on five datasets with seven baselines, five non-IID regimes, drift, heterogeneous links and outages, more than 2,000 seeded runs, and real-broker

fault-injection experiments.

To reach within 5 % of a centralised learner’s clustering cost, FedCAST needs 10–61 % fewer bytes than the best baseline on every dataset and up to $8.5\times$ fewer than periodic synchronisation. At matched budgets it ranks first ($p=2\times 10^{-4}$, Friedman). It detects emerging clusters $2.5\times$ faster than periodic updates while sending less. It recovers from a coordinator SIGKILL in about 1 s with state identical to an uninterrupted run, and loses no update during a 34 s broker outage.

2. Related work

Stream clustering. The two-phase paradigm, online micro-clusters followed by offline macro-clustering, was introduced by CluStream [1], building on BIRCH’s cluster features [8]. DenStream [2] added exponential decay and density-based macro-clustering, and DBSTREAM [9] a shared-density graph; see [10] for a survey. FedCAST uses decayed CFs as its unit of exchange because they are additive, cheap to send, and give the k-means cost in closed form, which is what makes an objective-linked trigger computable on the device.

Distributed stream clustering. Cormode et al. [3] maintain a continuous k-center clustering over distributed streams with approximation guarantees. DGClust [4] sends grid-cell state changes from sensors, and Tran and Sattler [5] send micro-clusters from remote sites to a coordinator. These works reduce communication with fixed, objective-agnostic rules (state change, period or threshold). They have no byte budget, do not adapt to link conditions, do not treat statistical heterogeneity, and are evaluated in simulation without a messaging substrate. Coreset-based methods [11] offer strong guarantees for static data but no cheap incremental delta.

Federated clustering. k-FED [6] performs one-shot federated k-means and shows that heterogeneity *helps* when each device holds few clusters. FFCM [12] federates fuzzy c-means, and AFCL [7] handles asynchrony and an unknown number of clusters. All of them assume static local datasets. We run k-FED periodically as a strong baseline and report the regimes in which it wins.

Federated learning on streams. FedCluLearn [13] uses micro-cluster indexing to fight forgetting in federated continual *supervised* learning. Silva et al. [14] and DFAS [15] address federated anomaly detection on streams. None of these targets clustering of the data itself under an explicit communication budget.

Event-triggered and communication-efficient learning. Communication efficiency is central in federated learning [16, 17]. Event-triggered schemes send when the norm of a parameter change exceeds a threshold, possibly adapted to device resources [18, 19]. FedCAST differs in three ways: its trigger is measured in the clustering objective and bounds the server’s error (Propositions 1 and 2); it enforces a byte budget with a guarantee (Proposition 3); and it prioritises *which* parts of the model to send.

Kafka for federated ML. Kafka-ML [20] and its federated extension [21] use Kafka as the data and model transport for neural-network training. In FedCAST, Kafka’s ordering, idempotence, replay and compaction semantics are part of the algorithm’s recovery and exactly-once-state argument, and we measure them under crashes.

3. Problem formulation

Setting. A set of m edge nodes observes unlabeled streams. Node i receives points $x \in \mathbb{R}^d$ drawn from a time-varying local distribution $P_i(t)$, and the local distributions are non-IID: $P_i \neq P_j$, and a cluster may be observed by a single node only. The network-wide distribution is the mixture $P(t) = \sum_i \pi_i(t) P_i(t)$. Raw data must not leave a node. Node i is connected to a Kafka cluster by a link whose cost per byte, the *price* $p_i(t) \geq 1$, varies with latency, loss and bandwidth. Node i may spend at most B_i bytes per budget window of length W .

Cluster features. Node i maintains a set $S_i(t)$ of at most q micro-clusters, each summarised by a time-decayed cluster feature (CF) [1, 2, 8]

$$\text{CF} = (n, \mathbf{LS}, SS, \tau), \quad (1)$$

$$n = \sum_x w_x, \quad \mathbf{LS} = \sum_x w_x x, \quad SS = \sum_x w_x \|x\|^2,$$

with weights $w_x = 2^{-(t-t_x)/H}$ for half-life H and last-update time τ . Decaying a CF to time t multiplies all three sums by the same factor $2^{-(t-\tau)/H}$. The centroid $\mu = \mathbf{LS}/n$ and the RMS radius are therefore invariant under decay, and a copy of a CF together with its τ determines its decayed value at any later time.

Cost of a CF. For a centre c , the k-means cost of all points summarised by a CF has the closed form

$$J(\text{CF}, c) = \sum_x w_x \|x - c\|^2 = SS - 2\mathbf{LS}^\top c + n\|c\|^2. \quad (2)$$

For a set of centres $C = \{c_1, \dots, c_k\}$ let $J(\text{CF}; C) = \min_j J(\text{CF}, c_j)$. The minimiser depends only on μ , because $J(\text{CF}, c) = n\|\mu - c\|^2 + \text{const}$.

Objective. The coordinator stores, for each node, the CFs it last received: $\hat{S}_i(t)$. From these it computes global centres $C(t)$. Let $C^\star(t)$ be the centres a centralised learner with all raw data would compute. We seek send policies that solve

$$\min \frac{1}{T} \sum_t [J(C(t); P(t)) - J(C^\star(t); P(t))] \quad \text{s.t.} \quad \frac{1}{T} \sum_t b_i(t) \leq \frac{B_i}{W} \quad \forall i, \quad (3)$$

where $b_i(t)$ is the number of bytes node i sends at time t . The gap in (3) has two sources: the macro-clustering itself, which any method incurs, and *staleness*, the difference between $\hat{S}_i$ and S_i . FedCAST controls the second source directly.

4. FedCAST

FedCAST has three components (Figure 1): an online CF micro-clusterer on every node, a budgeted and objective-linked send policy on every node, and a heterogeneity-aware aggregator at the coordinator. All communication goes through Kafka (Section 5).

4.1. Local summarisation

Each node runs a CluStream-style update [1] with DenStream-style decay [2]. A point is absorbed by the nearest micro-cluster if it lies within r_{mult} times that micro-cluster's RMS radius, where singletons use half the distance to their nearest neighbour and a floor r_{min} applies. Otherwise the point opens a new micro-cluster. When all q slots are full, the lightest micro-cluster is deleted if its decayed weight is below w_{min} ; otherwise the two closest micro-clusters are merged. We keep an exact nearest-neighbour cache between centroids, updated in $O(qd)$ per change, so a merge costs $O(q)$ rather than $O(q^2d)$. Every slot carries a modification counter. The node also keeps a copy of the CFs it has sent, $\hat{S}_i$, which is *exactly* the state held by the server, because both sides apply the same deterministic decay.

4.2. Objective-linked staleness

Pair the current and the server-side CFs of node i by micro-cluster identifier; a missing CF counts as zero. For every identifier define

$$e_{\text{id}} = |J(\text{CF}_{\text{id}}; C) - J(\widehat{\text{CF}}_{\text{id}}; C)| + \rho(\|\mathbf{LS}_{\text{id}} - \widehat{\mathbf{LS}}_{\text{id}}\| + \bar{c}_{\text{id}} |n_{\text{id}} - \hat{n}_{\text{id}}|), \quad (4)$$

where C is the latest global model received on the downlink and $\bar{c}_{\text{id}}$ is the larger norm of the two global centres the two copies are assigned to. The staleness of node i is $\Delta_i = \sum_{\text{id}} e_{\text{id}}$. A micro-cluster that has not changed since it was sent contributes exactly zero, and Δ_i costs $O(|\text{dirty}|kd)$ to compute.

Proposition 1 (Cost error). *For any centres C , $|J(S_i; C) - J(\hat{S}_i; C)| \leq \sum_{\text{id}} |J(\text{CF}_{\text{id}}; C) - J(\widehat{\text{CF}}_{\text{id}}; C)| \leq \Delta_i$. Consequently, the coordinator's estimate of the network-wide cost satisfies $|J(\hat{S}; C) - J(S; C)| \leq \sum_i \Delta_i$.*

Proposition 2 (Centroid displacement). *Let A_j be the set of identifiers assigned to centre j in both S_i and $\hat{S}_i$. The Lloyd updates $c'_j = \mathbf{L}_j/N_j$ computed from the current CFs and $\hat{c}'_j = \hat{\mathbf{L}}_j/\hat{N}_j$ computed from the server copies, with $\mathbf{L}_j = \sum_{A_j} \mathbf{LS}_{\text{id}}$ and $N_j = \sum_{A_j} n_{\text{id}}$, satisfy*

$$\|c'_j - \hat{c}'_j\| \leq \frac{1}{N_j} \sum_{\text{id} \in A_j} (\|\mathbf{LS}_{\text{id}} - \widehat{\mathbf{LS}}_{\text{id}}\| + \|\hat{c}'_j\| |n_{\text{id}} - \hat{n}_{\text{id}}|) \leq \frac{\Delta_i}{\rho N_j}$$

whenever $\|\hat{c}'_j\| \leq \bar{c}_{\text{id}}$. This holds in particular at a Lloyd fixed point $\hat{c}'_j = c_j$, which is where a warm-started coordinator operates.

Proofs are in Appendix A. Proposition 1 bounds the server's error in the k-means objective by the staleness. Proposition 2 bounds how far the global centres can move because of staleness. Micro-clusters that switch to a different centre are captured by the cost term.

4.3. Budgeted, prioritised sending

Once per tick, node i runs Algorithm 1.

Prioritisation. Sort the dirty micro-clusters by e_{id} and pick the smallest prefix that, together with all pending deletions, covers a fraction γ of Δ_i . This yields a payload of b bytes that covers $D_c \geq \gamma \Delta_i$. Because every CF record has the same size, this greedy choice is the optimal fractional knapsack. The remaining small changes are not lost: they accumulate and are sent once they matter.

Trigger. Send if

$$D_c \geq \lambda_i p_i b \quad \text{or} \quad \text{novelty}_i \geq \max(v_{\text{min}}, v N_i), \quad (5)$$

provided the token bucket holds b tokens. The bucket refills at rate B_i/W and has capacity $\kappa_i = \max(B_i, s_{\text{full}})$, where s_{full} is the size of a full summary. The novelty mass is the weight of new micro-clusters that lie farther than $3\times$ the radius from every global centre: a concept the server has not seen yet. This override is what makes node-exclusive clusters appear quickly.

Dual ascent. At the end of each window the multiplier is updated multiplicatively,

$$\lambda_i \leftarrow \lambda_i \exp(\eta \text{clip}_{[-1,2]}(\frac{\text{spent}_i - B_i}{B_i})), \quad (6)$$

which is projected subgradient ascent on $\log \lambda_i$ for the Lagrangian of (3). The update is scale-free: λ carries the units of cost per byte, but only ratios enter (6), so the same η works on every dataset. λ_i is initialised to the first observed ratio $D_c/(p_i b)$.

Proposition 3 (Budget). *(i) Hard: over any interval of length T , node i sends at most $\kappa_i + B_i T/W$ bytes. (ii) Long-run: if staleness is bounded ($\Delta_i \leq \Delta_{\text{max}}$), messages have at least b_{min} bytes, $p_i \geq 1$, and the novelty override is disabled, then after n windows $\frac{1}{n} \sum_{w \leq n} \min(\frac{\text{spent}_w - B_i}{B_i}, 2) \leq \frac{\log(\lambda^\dagger e^{2\eta}/\lambda_0)}{\eta n}$ with $\lambda^\dagger = \Delta_{\text{max}}/b_{\text{min}}$. The average spend therefore converges to at most B_i at rate $O(1/n)$.*

Link price. p_i is an exponentially weighted estimate of (wire bytes/payload bytes) $\cdot (1 + \text{latency}/\ell_{\text{ref}})$. Retransmissions and slow links therefore make bytes more expensive, and such nodes send only higher-value updates. Both quantities are available on a real device from the producer's delivery reports.

4.4. Heterogeneity-aware aggregation

The coordinator keeps, per node, the CFs it has received and applies each delta exactly once: a summary with sequence number $\leq$ the last applied one is ignored. Every T_{agg} it (i) decays all stored CFs to the current time, so that silent nodes fade out instead of freezing stale mass into the model; (ii) optionally rescales node i 's total mass N_i to N_i^β so that a single fast node cannot dominate; and (iii) runs weighted k-means over the CF centroids, warm-started from the previous model and followed by *swap local search* [22]. The swap step moves the centre with the smallest Ward merge cost to the worst-served micro-cluster and keeps the move only if the total cost decreases. Warm starts keep cluster identities stable over time but get stuck when a cluster appears at a single node: the new cluster is absorbed by its nearest large neighbour. The swap step removes exactly this failure mode. A second k-means++ [23] restart guards against poor local optima. The resulting model (centres, radii, weights) is published to a compacted Kafka topic.

Algorithm 1 FedCAST edge node i (one tick at time t)

```

1: ingest new points into the CF micro-clusters; prune stale ones
2: refill token bucket:  $\text{tok} \leftarrow \min(\kappa, \text{tok} + \Delta t B/W)$ 
3: if no global model yet then send full summary (bootstrap)
4: end if
5: compute  $e_{\text{id}}$  for dirty ids and deletions (Eq. 4);  $\Delta \leftarrow \sum e$ 
6:  $P \leftarrow$  smallest top- $e$  prefix with coverage  $D_c \geq \gamma\Delta$ ;  $b \leftarrow$  bytes of  $P$ 
   + deletions
7: if ( $D_c \geq \lambda p b$  or novelty) and  $\text{tok} \geq b$  then
8:   produce  $\text{delta}(P, \text{deletions}, \text{seq})$  to  $\text{fsc.summaries}$  with key
   node- $i$ 
9:   update server copy  $\hat{S}$ ;  $\text{tok} \leftarrow \text{tok} - b$ ;  $\text{seq} \leftarrow \text{seq} + 1$ 
10: end if
11: on delivery report: update price  $p$ ; on new  $\text{fsc.global}$  record:
   update  $C$ 
12: at window end: update  $\lambda$  with Eq. (6)

```

Table 1: Kafka semantics used by FedCAST.

Semantics	Role in FedCAST
Per-partition order, key = node	Deltas of a node are applied in the order produced; a delta may assume its predecessor.
Idempotent producer + per-node sequence numbers	Retries on lossy links never double-apply a delta; replays after a crash are ignored.
Replayable log + committed offsets	A restarted coordinator replays only what it had not yet checkpointed; nodes never resend.
Log compaction	<code>fsc.snapshots</code> is a changelog holding one record per node (the server-side CFs and the offsets they reflect); <code>fsc.global</code> always holds the latest model, also for late joiners.

Complexity. Per point: $O(qd)$. Per tick and node: $O(|\text{dirty}|(kd + \log q))$. Per aggregation: $O(I m q k d)$ for I Lloyd iterations, over at most $m q$ micro-clusters and never over raw points. Uplink messages cost $20 + |P|(24 + 4d) + 4|\text{deletions}|$ bytes plus about 24 bytes of Kafka record overhead.

5. System design on Apache Kafka

Kafka [24, 25] is not only the transport in FedCAST. Four of its semantics are part of the algorithm’s correctness argument (Table 1).

Topics. `fsc.summaries` (8 partitions, key node- i) carries deltas. `fsc.snapshots` (compacted) is the coordinator’s changelog. `fsc.global` (compacted, one partition) carries the model. `fsc.raw` is used only by the centralised baseline, and `fsc.metrics` carries telemetry.

Wire format. Messages use a fixed little-endian binary layout: a 20-byte header (kind, node, sequence number, time, counts, dimension) followed by packed arrays. The size of a pending delta is therefore known *before* encoding, which the trigger (5) requires, and decoding is zero-copy. JSON would be 3–5 $\times$ larger

and of variable size. Avro or Protobuf would require a schema registry service.

Exactly-once state and recovery. The coordinator follows the state-store-plus-changelog pattern of stream processors [26]. Every few seconds it writes, per node, a full CF record to `fsc.snapshots`, with the last applied sequence number and the `fsc.summaries` offsets as record headers, and then commits the offsets. On restart it reads the compacted changelog, restores all node states, and resumes `fsc.summaries` at the checkpointed offsets. Records after the checkpoint are applied again, and any record at or below a node’s restored sequence number is ignored. The recovered state is therefore identical to that of an uninterrupted coordinator (Section 7.5).

Low-memory deployment. The broker runs in KRaft mode (no ZooKeeper) with a 256 MB heap. We measured 317 MB idle and 423 MB peak RSS under a 300k-message load test; in Docker with a 512 MB cap it never ran out of memory. All simulated nodes share one Python process (≈ 150 MB) with one idempotent producer each, and the coordinator needs ≈ 140 MB. The complete system runs in under 1 GB of RAM, which makes it reproducible on commodity laptops without Flink or Spark.

Two execution modes, one code base. The same `EdgeNode`, policy, codec and coordinator classes run (i) against a real broker and (ii) inside a deterministic discrete-event simulator. The simulator replaces the broker with seeded per-node links (delay, jitter, bandwidth cap, loss with retransmission, outages) that preserve per-key FIFO order and count retransmitted bytes. The simulator enables thousands of seeded runs. The Kafka mode provides the systems measurements and validates the simulator’s byte accounting against Kafka’s producer statistics.

6. Experimental setup

Research questions. **RQ1** How much communication does FedCAST save at equal clustering quality, and what quality does it achieve at equal byte budgets? **RQ2** How does quality depend on the degree of statistical heterogeneity? **RQ3** How fast does the global model react to drift and to newly emerging clusters? **RQ4** How robust is the system to heterogeneous links, outages, and coordinator or broker failures? **RQ5** How does it scale with the number of nodes?

Datasets. Labels are used for evaluation only. *SynDrift* is our seeded evolving stream: 60k points in $d=10$ from $k=15$ anisotropic Gaussians with random orientation, per-axis scales in $[0.3, 1.5]$ and Dirichlet(2) sizes. Four clusters drift by 5 units, two clusters emerge at 35 % and 65 % of the stream, and one vanishes at 50 %. *NSL-KDD* [27]: network-intrusion records with 5 classes (normal, DoS, probe, R2L, U2R). We log-transform the 38 numeric features and one-hot encode protocol and flag, giving 51 features; we use a random sample of 60k of the 126k records. *Shuttle*, *Pen-Digits* and *Letter* come from PMLB [28]: 58k $\times$ 9 with 7 classes and 78 % majority, 11k $\times$ 16 with 10 classes, and 20k $\times$ 16 with 26 classes. Features are standardised.

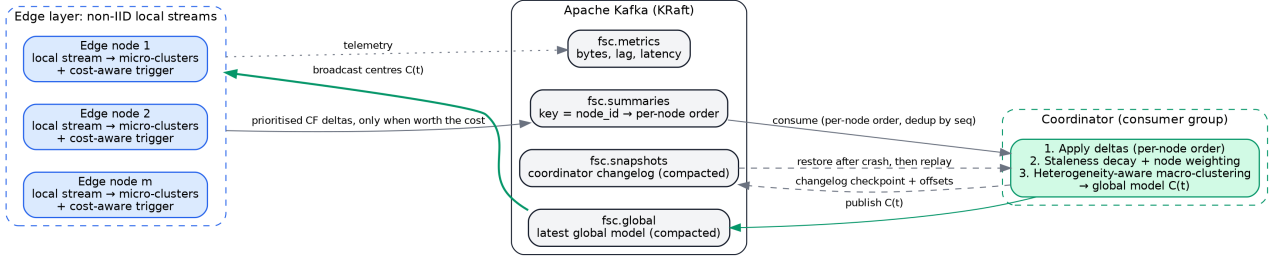


Figure 1: FedCAST over Apache Kafka. Edge nodes publish prioritised CF deltas only when they are worth their byte cost. The coordinator applies them in per-node order, checkpoints its state to a compacted changelog, and broadcasts the global model on a compacted topic that the edges use to compute staleness.

Streams and partitions. The data are split across $m=10$ nodes and replayed over 300 s of stream time with a 1 s tick. Partitions: IID; Dirichlet(α) label skew [29] with $\alpha \in \{1, 0.3, 0.1\}$ (default 0.3); cluster-exclusive nodes that hold $k' \in \{2, 1\}$ classes (the regime of [6]); and *class-rotation drift*, where at 50 % of the stream every node switches to the classes of its neighbour.

Network. Nodes alternate between a WAN link (80 ms delay, 1 % loss, 5 Mbit/s) and a cellular link (150 ms, 3 %, 1 Mbit/s). RQ4 adds LAN and poor links (300 ms, 8 %, 128 kbit/s) and 45 s outages on three nodes. Lost messages are retransmitted after a 300 ms timeout, and the retransmitted bytes are counted as wire bytes.

Baselines. All baselines share FedCAST’s local micro-clusterer and aggregator, so differences come from *what and when* each node sends: *Centralised-Raw* streams every point through Kafka to one server-side micro-clusterer with m_q slots, giving the quality upper bound; *Naive* sends the full summary whenever it changed; *Periodic* sends the full summary every T seconds, as in classic federated rounds [16]; *Periodic- Δ* sends only the changed micro-clusters every T seconds; *Change-Threshold* sends micro-clusters that were created, deleted, or changed by more than ε relative to their radius or weight [4, 5]; *Norm-Trigger* sends changed micro-clusters when the relative norm of the summary change exceeds δ [18]; *k-FED* [6] sends k' local centres, as exact CF sums, every T seconds, with the oracle number of local classes as k' . We sweep each method’s knob: $T \in \{15, \dots, 240\}$ s, $\varepsilon \in \{0.3, 1, 3\}$, $\delta \in \{0.05, 0.2, 0.5\}$, and FedCAST budgets $B/W \in \{5, \dots, 500\}$ B/s per node.

FedCAST settings. $q=50$, $H=120$ s, $r_{\text{mult}}=2$, $r_{\text{min}}=0.25$; $\gamma=0.9$, $\rho=1$, $\eta=0.5$, $W=10$ s; aggregation every 5 s with 3 swaps and 2 restarts; k equals the number of classes. The same values are used for every dataset; sensitivity is reported in Section 7.4.

Metrics. Every 10 s the server’s current model labels the points of the last 60 s, up to 3000 points. We report the *cost ratio*, the k-means cost of the server model divided by that of Centralised-Raw on the same seed (the objective FedCAST controls; lower is better), together with ARI [30], NMI, purity and per-class recall. Cost is measured in uplink bytes: payload plus a 24-byte per-record Kafka overhead, which is validated against Kafka’s producer statistics in Section 7.5. Averages exclude the first 10 % of the stream.

Table 2: Uplink kB needed to reach a cost ratio ≤ 1.05 (mean curve, log-interpolated). “–”: not reached.

	FedCAST	Per.	Per.- Δ	k-FED	Chg.	Norm	saving
SynDrift	44	372	256	113	574	276	61 %
NSL-KDD	362	668	484	–	969	907	25 %
Shuttle	133	–	–	–	263	242	45 %
Pen-Digits	48	169	140	53	309	309	10 %
Letter	83	334	256	204	435	412	60 %

Statistical protocol. Each configuration is run with 5 seeds, which vary the partition, the stream order, the links and the aggregation. For budget-matched comparisons we interpolate every method’s (bytes, quality) curve per seed at 1 %, 3 % and 10 % of Centralised-Raw’s bytes, in log-bytes, keeping the best quality attainable within the budget. A method that cannot operate at a budget is marked infeasible. We use the Friedman test with the Nemenyi critical difference over dataset×budget blocks, and one-sided Wilcoxon signed-rank tests with Holm correction [31].

Implementation. Python 3.11, NumPy, scikit-learn and confluent-kafka (librdkafka); Apache Kafka 3.8 in KRaft mode. All runs used one 4-core machine; the complete system also runs on a laptop with under 1 GB of RAM.

7. Results

7.1. RQ1: Communication versus quality

Figure 2 shows the Pareto fronts. FedCAST’s front lies at or below every baseline from about 1.5 % of the Centralised-Raw traffic upward, on all five datasets. Table 2 quantifies the gap as the bytes each method needs to bring the cost within 5 % of the centralised learner. FedCAST needs **10–61 % fewer bytes than the best baseline** on each dataset, and **1.8–8.5× fewer than Periodic**. Some baselines never reach this quality within their parameter range: k-FED on NSL-KDD and Shuttle, and Periodic on Shuttle.

Table 3 compares methods at equal budgets. Among the methods that can operate at small budgets (FedCAST, Periodic, Periodic- Δ , k-FED), FedCAST obtains the lowest mean Friedman rank over the 10 dataset×budget blocks where all four are feasible: 1.1, against 2.5 for k-FED, 2.8 for Periodic- Δ and 3.6

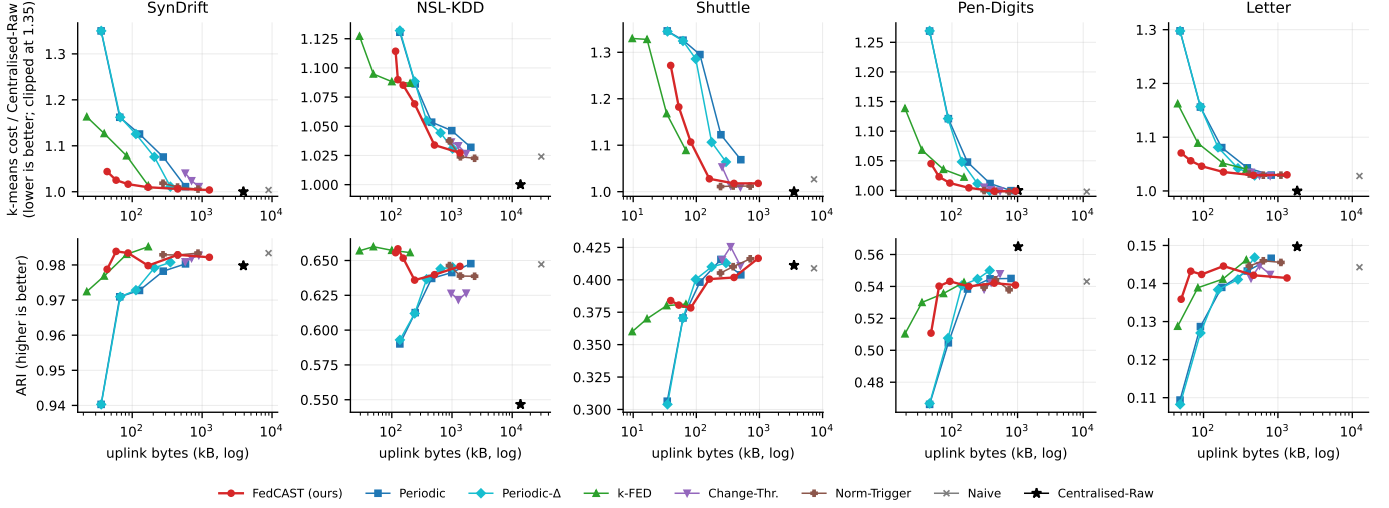


Figure 2: Cost-quality trade-off on five datasets (mean of 5 seeds; each marker is one parameter setting). Top: k-means cost of the server’s model relative to Centralised-Raw (lower is better). Bottom: ARI. FedCAST (red) dominates across the practically relevant range. k-FED is cheaper at the extreme low end on well-separated data. Change- and Norm-triggered reporting cannot operate below a data-dependent byte floor.

Table 3: Budget-matched cost ratio (lower is better; mean of 5 seeds) at 1/3/10 % of Centralised-Raw bytes. “–”: the method cannot operate within that budget. Rows where no method is feasible are omitted.

Data	Bud.	FedCAST	Per.	Per-Δ	k-FED	Chg.	Norm
SynDrift	1%	–	2.796	2.813	1.128	–	–
	3%	1.013	1.130	1.123	1.047	–	–
	10%	1.007	1.046	1.011	1.013	–	1.012
NSL-KDD	1%	1.084	1.157	1.158	1.082	–	–
	3%	1.043	1.059	1.053	1.078	–	–
	10%	1.027	1.039	1.031	1.078	1.030	1.025
Shuttle	1%	–	1.354	1.355	1.161	–	–
	3%	1.076	1.297	1.263	1.089	–	–
	10%	1.012	1.095	1.064	1.089	1.013	1.003
Pen-Dig.	3%	–	–	–	1.084	–	–
	10%	1.011	1.105	1.097	1.030	–	–
Letter	3%	1.067	1.270	1.271	1.139	–	–
	10%	1.036	1.079	1.072	1.052	–	–

for Periodic. The Friedman test gives $\chi^2=19.6$, $p=2.1 \times 10^{-4}$ (Figure 3). One-sided Wilcoxon tests over all paired seeds confirm the ordering after Holm correction: FedCAST beats Periodic ($p=3.5 \times 10^{-13}$, wins 96 % of pairs), Periodic-Δ ($p=5.4 \times 10^{-11}$, 94 %) and k-FED ($p=1.7 \times 10^{-9}$, 88 %). At 10 % of raw traffic, the change- and norm-triggered baselines become feasible and are within about 1 % of FedCAST, but they need 3–20× more bytes to get there (Table 2). Two regimes favour k-FED: the very lowest budgets (1 %) and the small, well-separated Pen-Digits stream. There, a single message of k' centres is cheaper than FedCAST’s bootstrap summary of q micro-clusters. This is a genuine limitation that we discuss in Section 8.

Byte counts are credible. On a real broker, our per-message estimate (payload plus 24 B record overhead) exceeded librdkafka’s `txmsg_bytes` by only 0.7–3.0 % for every method (Table 7), so all reported savings are conservative.

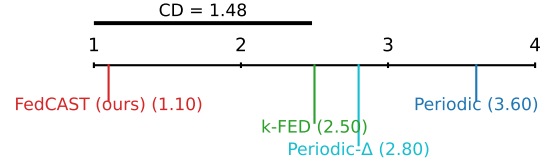


Figure 3: Critical-difference diagram (Nemenyi, $\alpha=0.05$) of mean ranks by budget-matched cost ratio.

7.2. RQ3: Drift and emerging clusters

On SynDrift (Figure 4, left), the novelty override and the objective-linked trigger let FedCAST report each emerging cluster as soon as enough mass has accumulated. The mean delay until an emerging class is recalled at $\geq 50\%$ is 10.9 ± 5.6 s at 10 B/s and 9.4 ± 2.6 s at 20 B/s, against 27.4 s for Periodic-60, Periodic-120, Periodic-Δ and k-FED. It does so with 59–87 kB of uplink, compared with 113–127 kB for the 60 s periodic variants. Change-Threshold reacts within one evaluation interval (4.9 s) but spends 574 kB, almost 10× more. The cost ratio over the whole evolving stream is 1.027–1.043 for FedCAST, against 1.23–1.27 for the periodic methods and k-FED. Under class-rotation drift, which changes *which node* sees a class but not the global mixture, all methods keep a valid model. FedCAST again pairs the lowest traffic with a lower cost than Periodic (Pen-Digits: 1.013 at 96 kB against 1.042 at 173 kB).

The budget is respected. Figure 5 shows every node’s cumulative spend. After the one-time bootstrap summary, which is paid from the token bucket’s initial allowance κ , the slope equals the configured budget: 10.0 and 18 B/s for $B/W=10$ and 20. No node ever crosses the hard envelope $\kappa + Bt$ of Proposition 3.

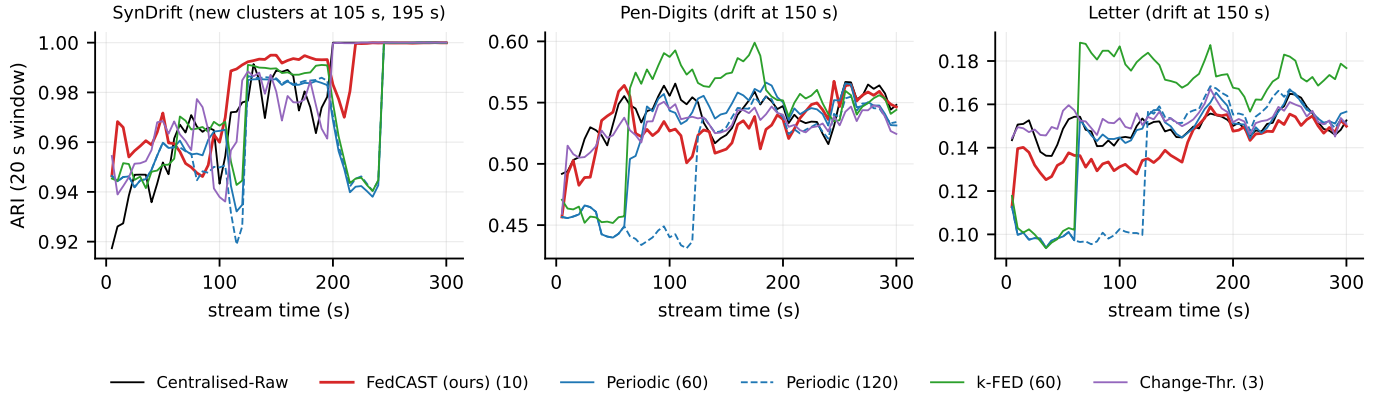


Figure 4: ARI over time (20 s window, mean of 5 seeds). On SynDrift, new clusters emerge at 105 s and 195 s at few nodes. FedCAST (10 B/s per node, 59 kB total) picks them up about 2.5× faster than Periodic and k-FED, while sending less than Periodic-60 (127 kB). On Pen-Digits and Letter, node class sets rotate at 150 s.

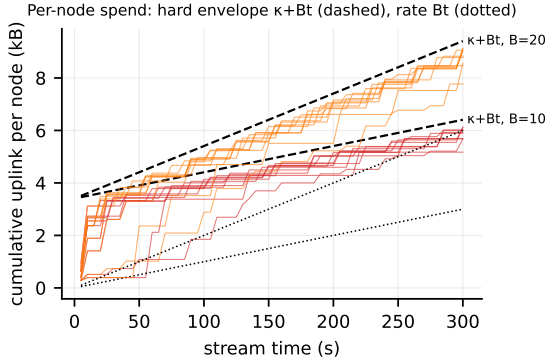


Figure 5: Per-node cumulative uplink for FedCAST on SynDrift with $B/W = 10$ and 20 B/s (all 10 nodes). Dashed: the hard envelope $\kappa + Bt$; dotted: the rate Bt .

7.3. RQ2: Degree of statistical heterogeneity

Figure 6 varies the partition from IID to cluster-exclusive nodes. FedCAST attains the lowest cost ratio in all 18 combinations of dataset and partition: 1.008–1.065, against 1.047–1.126 for Periodic and Periodic- Δ and 1.067–1.281 for k-FED. Its quality does not degrade with heterogeneity. On SynDrift it even improves, from 1.019 (IID) to 1.008 (one class per node), because exclusive clusters produce concentrated, high-value deltas.

Aggregator. Swap local search lowers the cost in 17 of 18 settings (e.g. Pen-Digits Dir(0.1): 1.016 against 1.037 without swaps). Node balancing ($\beta=0.5$) is neutral under mild skew but helps in the cluster-exclusive regime (Pen-Digits Excl(1): 1.011 against 1.029; Letter Excl(2): 1.027 against 1.038). There, a node with a large share of the traffic would otherwise pull several centres towards its own clusters.

k-FED’s regime. With one or two classes per node and the oracle k' , k-FED reaches the highest *label* agreement (Pen-Digits Excl(1): ARI 0.66 against 0.52–0.57 for all other methods, including Centralised-Raw), consistent with its theory [6]. Its k-means cost, however, is 22 % above the centralised learner.

Table 4: Ablation of FedCAST components at 20 B/s per node: cost ratio (lower is better, mean of 5 seeds) and mean uplink.

Variant	Syn	KDD	Shut.	Pen	Let.	kB
FedCAST	1.016	1.085	1.107	1.012	1.046	102
w/o prioritisation ($\gamma=1$)	1.333	1.132	1.331	1.035	1.061	87
full summaries	1.295	1.132	1.321	1.037	1.054	89
w/o swap search	1.019	1.101	1.218	1.026	1.057	102
w/o dual ascent	1.019	1.088	1.108	1.011	1.047	102
w/o link price	1.017	1.090	1.065	1.012	1.044	103
w/o novelty	1.017	1.090	1.092	1.012	1.047	102
w/o server decay	1.017	1.089	1.106	1.011	1.047	102

Each local centre equals a class mean, so k-FED recovers the class partition rather than the k-means optimum. When labels are the goal and each node holds few, well-separated classes, k-FED’s one-centre-per-class summaries are an excellent choice. When the goal is a faithful global clustering objective, FedCAST is.

7.4. Ablations

Table 4 isolates each component at a fixed budget. *Prioritised partial deltas are the single most important ingredient.* Sending all dirty micro-clusters ($\gamma=1$) or full summaries raises the cost by up to 32 % on SynDrift and 22 % on Shuttle. Under the same budget, large messages drain the token bucket, so sends become rare. Prioritisation spends the same bytes on the few micro-clusters that actually move the objective. Swap search is the second most important component (Shuttle: 1.107 against 1.218 without it). The remaining components leave the *average* cost at this budget essentially unchanged, because their purpose lies elsewhere. The novelty override targets *reaction time* to new clusters, the link price targets *wire cost* on bad links, and dual ascent targets *loose* budgets, where it prevents spending bytes on updates that carry no value. Section 7.5 and Table 6 evaluate these targeted effects. Removing the link price even lowers the cost slightly on Shuttle and Letter: the price deliberately shifts traffic away from expensive links at a small quality cost.

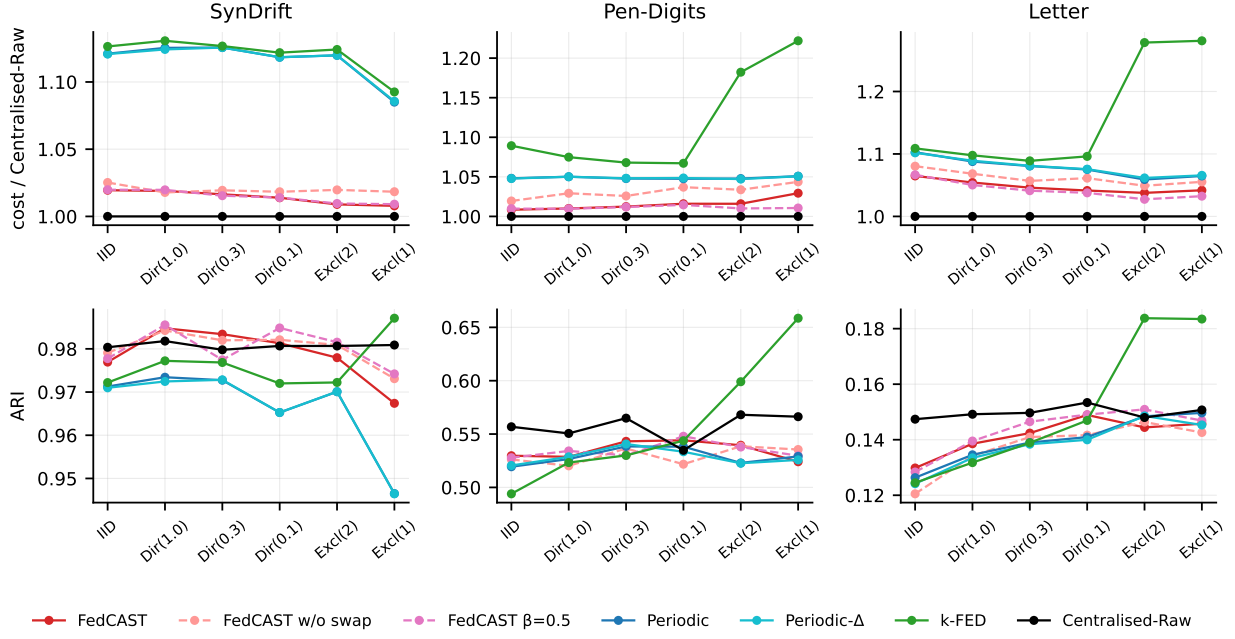


Figure 6: Quality as heterogeneity grows, from IID through Dirichlet label skew to cluster-exclusive nodes (Excl(k'): each node holds k' classes). FedCAST at 20 B/s per node against Periodic-60, Periodic- Δ -60 and k-FED-60 (similar or larger traffic), with two aggregator variants. Mean of 5 seeds.

Table 5: Cost ratio (uplink kB) under network scenarios; mean of 5 seeds.

Data	Method	homog. WAN	hetero	hetero+outage
SynDrift	FedCAST	1.015 (87)	1.016 (87)	1.017 (85)
	Periodic-60	1.125 (127)	1.125 (127)	1.126 (127)
	Periodic- Δ -60	1.126 (113)	1.125 (113)	1.126 (113)
	k-FED-60	1.127 (40)	1.126 (40)	1.127 (40)
	Change-Thr.	1.040 (574)	1.040 (574)	1.041 (574)
NSL-KDD	FedCAST	1.071 (154)	1.077 (156)	1.080 (156)
	Periodic-60	1.048 (459)	1.050 (459)	1.058 (459)
	Periodic- Δ -60	1.048 (379)	1.048 (379)	1.058 (379)
	k-FED-60	1.084 (49)	1.085 (49)	1.084 (49)
	Change-Thr.	1.028 (969)	1.027 (969)	1.027 (969)

7.5. RQ4: Network heterogeneity, outages and failures

Heterogeneous links and outages. Table 5 runs every method under three network scenarios: homogeneous WAN; heterogeneous LAN/WAN/cellular/poor links; and the same with 45 s outages at three nodes. Kafka-style retries and buffering deliver every summary eventually, and the coordinator’s staleness decay stops silent nodes from freezing stale mass. As a result, every method’s quality is nearly unchanged across scenarios. FedCAST’s cost ratio moves by at most 0.009 (SynDrift 1.015–1.017, NSL-KDD 1.071–1.080), and retransmissions add only 1–3 % wire bytes. The ordering between methods is the same as in RQ1.

What the remaining components are for. Table 6 tests the three components whose average-cost effect in Table 4 was negligible, each on the metric it is designed for. (i) *Link price*. Under loose budgets on heterogeneous links, the price shifts 4–7 % of the

Table 6: Targeted ablations: each component on the metric it is designed for (mean of 5 seeds).

Component	Setting	on / off	target metric (on vs. off)
Link price	Syn, 500 B/s	1276 / 1321 kB	bad/good-link bytes 0.93 vs 0.97
	KDD, 500 B/s	1371 / 1414 kB	bad/good-link bytes 0.99 vs 1.04
Dual ascent	Syn, 500 B/s	1.004 / 1.007	budget used 85 % vs 36 %
	Pen, 500 B/s	0.999 / 0.999	budget used 62 % vs 30 %
Novelty	Syn, 10 B/s	1.051 / 1.060	detection 10.9 s vs 10.9 s
	Syn, 20 B/s	1.035 / 1.035	detection 9.4 s vs 9.4 s

traffic away from cellular and poor links and saves 3 % of wire bytes at unchanged quality. At tight budgets (Table 5) it has no effect, because the token bucket, not the threshold, is the binding constraint. (ii) *Dual ascent*. Under a loose budget it makes the spend track the budget (62–85 % used, against 30–36 % with λ frozen at its initial value) without changing quality. This shows it is the mechanism that makes B , rather than an arbitrary λ_0 , the operating knob. It also shows that beyond some budget, extra bytes buy nothing, which suggests a value floor as a refinement. (iii) *Novelty override*. Detection delays are identical with and without it (10.9 and 9.4 s). The objective-linked score alone already reacts to emerging clusters, because a new micro-cluster far from every centre carries a large cost term. The override is therefore a redundant safety net, and we keep it only for its small cost benefit at the tightest budget (1.051 against 1.060).

Coordinator crash (real Kafka). We ran 10 edge nodes with FedCAST against two coordinators in different consumer groups: a *shadow* that never fails and a *victim* that we kill with SIGKILL after 15 s (no clean shutdown, no final checkpoint) and restart 8 s later. The restarted victim rebuilt all 10 node states from the

Table 7: Byte accounting on a real broker: our per-message estimate (payload + 24 B) against librdkafka’s `txmsg_bytes`.

Method	estimate (B)	Kafka (B)	difference
FedCAST 20 B/s	55,978	54,340	+3.0 %
FedCAST 150 B/s	230,798	226,028	+2.1 %
Periodic 30 s	189,160	187,900	+0.7 %
Periodic- Δ 30 s	153,264	152,004	+0.8 %
k-FED 30 s	57,760	56,500	+2.2 %
Centralised-Raw	2,486,473	2,450,275	+1.5 %

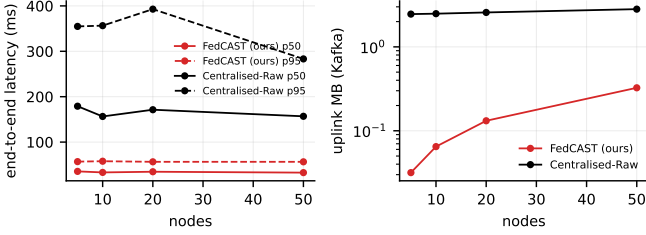


Figure 7: Real Kafka, 5–50 nodes on one 4-core machine: end-to-end latency from produce to coordinator consumption (left) and total uplink bytes (right, log scale).

compacted changelog in 1.08 s, resumed `fsc.summaries` at the checkpointed offsets, and caught up within 0.5 s. At the end of the run its state was *identical* to the shadow’s: the same micro-cluster identifiers for every node and a maximum node-mass difference of 0.0. All 238 summaries were applied exactly once, and no node resent anything.

Broker outage (real Kafka). We stopped the Kafka broker in the middle of a run for 34 s. The idempotent producers buffered and retried, and after the restart all 231 summaries sent by the nodes were applied by the coordinator: zero lost, zero delivery errors.

7.6. RQ5: Scalability and footprint

Figure 7 scales the real deployment from 5 to 50 edge nodes, each with its own idempotent producer, streaming 60k points in 20 s of wall time (about 1,900 points/s). FedCAST’s end-to-end latency stays flat at 33–35 ms (p50) and 57 ms (p95) at every scale, because nodes send few and small messages. Centralised-Raw’s latency is 4–5 $\times$ higher (156–179 ms p50, 283–393 ms p95) while it ships 2.5–2.8 MB, against 32–325 kB for FedCAST. FedCAST’s traffic grows linearly with the number of nodes (6.5 kB per node for 200 s of stream), as the per-node budget prescribes. Quality is unchanged (ARI 0.97–0.99). The simulated scaling experiment agrees: bytes per node are constant and ARI stays within 0.97–0.99 from 5 to 50 nodes (Periodic: 0.95–0.98). Memory stays small. At 50 nodes the edge process needs 172 MB, the coordinator 141 MB and the broker 475 MB, 788 MB in total, which confirms that the complete system runs on a commodity laptop.

8. Discussion

When to use which method. FedCAST pays off whenever the byte budget is tight relative to how fast the data evolves: it

spends bytes where the global objective is affected and when it is affected. When budgets are generous (above roughly 10 % of the bytes of sending raw data), every event-driven method converges to the centralised quality, and the choice among them matters little. k-FED is extremely frugal when clusters are well separated and each node holds few of them, exactly the regime its theory covers [6]. It needs the local k' (we gave it the oracle value), carries no shape information, and reacts only at its period, so it degrades on overlapping, drifting or imbalanced data. Change- and norm-triggered reporting reaches good quality but cannot be run below a data-dependent byte floor, and its thresholds must be re-tuned per dataset. FedCAST instead takes the budget itself as input.

What matters in FedCAST. The ablations separate the essential from the auxiliary. Prioritised partial deltas, together with an objective-linked score to rank them, deliver most of the gain: the budget is spent on the few micro-clusters that move the objective. Swap search matters under heterogeneity. The dual-ascent threshold turns the byte budget into the operating knob. The link price redistributes traffic only when budgets are loose. The novelty override turned out to be redundant with the objective-linked score. We report this negative result because it simplifies deployment: the score alone suffices to react to emerging clusters.

Kafka as part of the algorithm. Designing the protocol around Kafka’s semantics removed whole classes of failure handling from the algorithm. Deltas can assume their predecessor because of per-key order. Retransmissions are harmless because of idempotence and sequence numbers. Nodes never resend after a server crash because of replay and a compacted changelog. Late joiners and restarted nodes obtain the model from the compacted `fsc.global`. The price is a broker, which runs in about 400 MB.

Privacy. CFs reveal aggregate statistics (counts, sums, sums of squares) rather than records. They are not differentially private. Adding calibrated Gaussian noise to each delta after clipping $\|x\|$ is a natural extension [32]. The budget controller then also bounds the number of releases, and hence the privacy composition. We leave this three-way trade-off between bytes, accuracy and privacy to future work.

Limitations and threats to validity. (i) Scale: experiments emulate up to 50 nodes on one machine, with real Kafka for systems metrics and a validated simulator for sweeps. Wide-area deployments add effects such as cross-region replication that we do not model. (ii) Label-based metrics on real data are bounded by how well classes form k-means clusters (e.g. Shuttle, Letter). We therefore report the cost ratio to a centralised learner, which is the objective every method optimises. (iii) The macro-clustering assumes k is known; an unknown- k aggregator such as AFCL [7] or density-based aggregation could replace it without changing the send policy. (iv) Data rates are replayed in stream time, so wall-clock effects of real devices (CPU contention, energy) are measured only through the Kafka experiments. (v) At the very

lowest budgets (about 1 % of raw traffic), FedCAST cannot operate, because its bootstrap summary of q micro-clusters exceeds the budget, while k-FED's k' centres fit. Letting the trigger operate on compressed summaries (fewer, merged micro-clusters) would close this gap.

Code and data availability

All code, experiment configurations, raw results and scripts that regenerate every figure and table are available in the project repository. All datasets are public.

References

- [1] C. C. Aggarwal, J. Han, J. Wang, P. S. Yu, A framework for clustering evolving data streams, in: Proceedings of the 29th International Conference on Very Large Data Bases (VLDB), 2003, pp. 81–92.
- [2] F. Cao, M. Ester, W. Qian, A. Zhou, Density-based clustering over an evolving data stream with noise, in: Proceedings of the SIAM International Conference on Data Mining (SDM), 2006, pp. 328–339.
- [3] G. Cormode, S. Muthukrishnan, W. Zhuang, Conquering the divide: Continuous clustering of distributed data streams, in: Proceedings of the IEEE 23rd International Conference on Data Engineering (ICDE), 2007, pp. 1036–1045.
- [4] J. Gama, P. P. Rodrigues, L. Lopes, Clustering distributed sensor data streams using local processing and reduced communication, Intelligent Data Analysis 15 (1) (2011) 3–28.
- [5] D.-H. Tran, K.-U. Sattler, Communication-efficient exact clustering of distributed streaming data, in: Computational Science and Its Applications (ICCSA), Vol. 7975 of LNCS, 2013, pp. 421–436.
- [6] D. K. Dennis, T. Li, V. Smith, Heterogeneity for the win: One-shot federated clustering, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of PMLR, 2021, pp. 2611–2620.
- [7] Y. Zhang, Y. Zhang, Y. Lu, M. Li, X. Chen, Y.-m. Cheung, Asynchronous federated clustering with unknown number of clusters, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 39, 2025.
- [8] T. Zhang, R. Ramakrishnan, M. Livny, BIRCH: an efficient data clustering method for very large databases, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 1996, pp. 103–114.
- [9] M. Hahsler, M. Bolaños, Clustering data streams based on shared density between micro-clusters, IEEE Transactions on Knowledge and Data Engineering 28 (6) (2016) 1449–1461.
- [10] A. Zubároğlu, V. Atalay, Data stream clustering: a review, Artificial Intelligence Review 54 (2021) 1201–1236.
- [11] M.-F. Balcan, S. Ehrlich, Y. Liang, Distributed k-means and k-median clustering on general topologies, in: Advances in Neural Information Processing Systems (NeurIPS), 2013, pp. 1995–2003.
- [12] M. Stallmann, A. Wilbik, Towards federated clustering: A federated fuzzy c-means algorithm (FFCM), arXiv preprint arXiv:2201.07316 (2022).
- [13] M. Angelova, V. Boeva, S. Abghari, S. Ickin, X. Lan, FedCluLearn: Federated continual learning using stream micro-cluster indexing scheme, in: Machine Learning and Knowledge Discovery in Databases (ECML PKDD), 2025.
- [14] P. R. Silva, J. Vinagre, J. Gama, Federated anomaly detection over distributed data streams, arXiv preprint arXiv:2205.07829 (2022).
- [15] B. Li, L. Cheng, Z. Qin, Y. Wu, Density-aware and cluster-based federated anomaly detection on data streams, in: Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM), 2025.
- [16] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017, pp. 1273–1282.
- [17] P. Kairouz, H. B. McMahan, et al., Advances and open problems in federated learning, Foundations and Trends in Machine Learning 14 (1–2) (2021) 1–210.
- [18] S. Ghosh, B. Aquino, V. Gupta, EventGrad: Event-triggered communication in parallel machine learning, Neurocomputing 483 (2022) 474–487.
- [19] S. Zehtabi, S. Hosseinalipour, C. G. Brinton, Decentralized event-triggered federated learning with heterogeneous communication thresholds, arXiv preprint arXiv:2204.03726 (2022).
- [20] C. Martín, P. Langendoerfer, P. S. Zarrin, M. Díaz, B. Rubio, Kafka-ML: Connecting the data stream with ML/AI frameworks, Future Generation Computer Systems 126 (2022) 15–33.
- [21] A. J. Carnerero-Cano, C. Martín, M. Díaz, Towards flexible data stream collaboration: Federated learning in Kafka-ML, Internet of Things 25 (2024) 101036.
- [22] T. Kanungo, D. M. Mount, N. S. Netanyahu, C. D. Piatko, R. Silverman, A. Y. Wu, A local search approximation algorithm for k-means clustering, Computational Geometry 28 (2–3) (2004) 89–112.
- [23] D. Arthur, S. Vassilvitskii, k-means++: The advantages of careful seeding, in: Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2007, pp. 1027–1035.
- [24] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proceedings of the NetDB Workshop, 2011.
- [25] G. Wang, J. Koshy, S. Subramanian, K. Paramasivam, M. Zadeh, N. Narkhede, J. Rao, J. Kreps, J. Stein, Building a replicated logging system with Apache Kafka, Vol. 8, 2015, pp. 1654–1655.
- [26] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, K. Tzoumas, State management in Apache Flink: consistent stateful distributed stream processing, Vol. 10, 2017, pp. 1718–1729.
- [27] M. Tavallaei, E. Bagheri, W. Lu, A. A. Ghorbani, A detailed analysis of the KDD CUP 99 data set, IEEE Symposium on Computational Intelligence for Security and Defense Applications (2009).
- [28] R. S. Olson, W. La Cava, P. Orzechowski, R. J. Urbanowicz, J. H. Moore, PMLB: a large benchmark suite for machine learning evaluation and comparison, BioData Mining 10 (1) (2017) 36.
- [29] T.-M. H. Hsu, H. Qi, M. Brown, Measuring the effects of non-identical data distribution for federated visual classification, in: arXiv preprint arXiv:1909.06335, 2019.
- [30] L. Hubert, P. Arabie, Comparing partitions, Journal of Classification 2 (1) (1985) 193–218.
- [31] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine Learning Research 7 (2006) 1–30.
- [32] A. Epasto, T. Mukherjee, P. Zhong, Differentially private clustering in data streams, in: arXiv preprint arXiv:2307.07449, 2023.

Appendix A. Proofs

Proof of Proposition 1. Micro-clusters are assigned independently, so $J(S_i; C) = \sum_{id} J(\text{CF}_{id}; C)$, where an identifier absent from one of the two sets contributes 0 on that side. Pairing the two sums by identifier and applying the triangle inequality gives

$$\begin{aligned} |J(S_i; C) - J(\hat{S}_i; C)| &= \left| \sum_{id} (J(\text{CF}_{id}; C) - J(\widehat{\text{CF}}_{id}; C)) \right| \\ &\leq \sum_{id} |J(\text{CF}_{id}; C) - J(\widehat{\text{CF}}_{id}; C)|. \end{aligned}$$

Each summand is the first term of e_{id} in (4), and the second term is non-negative, so the sum is at most Δ_i . Summing over nodes gives the network-wide statement. Both copies are decayed to the same time with the same factor per identifier (Section 3), so an unchanged micro-cluster has $e_{id} = 0$. $\square$

Proof of Proposition 2.

$$\begin{aligned} c'_j - \tilde{c}'_j &= \frac{\mathbf{L}_j}{N_j} - \frac{\hat{\mathbf{L}}_j}{\hat{N}_j} = \frac{\mathbf{L}_j - \hat{\mathbf{L}}_j}{N_j} + \hat{\mathbf{L}}_j \left(\frac{1}{N_j} - \frac{1}{\hat{N}_j} \right) \\ &= \frac{\mathbf{L}_j - \hat{\mathbf{L}}_j}{N_j} + \tilde{c}'_j \frac{\hat{N}_j - N_j}{N_j}. \end{aligned}$$

Hence $\|c'_j - \hat{c}'_j\| \leq (\|\mathbf{L}_j - \hat{\mathbf{L}}_j\| + \|\hat{c}'_j\| |N_j - \hat{N}_j|)/N_j$. Because both sums run over the same identifiers A_j , the triangle inequality gives $\|\mathbf{L}_j - \hat{\mathbf{L}}_j\| \leq \sum_{A_j} \|\mathbf{L}\mathbf{S}_{\text{id}} - \widehat{\mathbf{L}}\mathbf{S}_{\text{id}}\|$ and $|N_j - \hat{N}_j| \leq \sum_{A_j} |n_{\text{id}} - \hat{n}_{\text{id}}|$. With $\|\hat{c}'_j\| \leq \bar{c}_{\text{id}}$, each summand is at most e_{id}/ρ . $\square$

Proof of Proposition 3. (i) A token bucket with rate B_i/W and capacity κ_i starts full, is never negative, and every send consumes its size. After time T the total consumed is at most $\kappa_i + B_i T/W$.

(ii) Let $\theta_w = \log \lambda_w$ and $g_w = \text{clip}_{[-1,2]}((\text{spent}_w - B_i)/B_i)$, so $\theta_{w+1} = \theta_w + \eta g_w$. Telescoping gives $\sum_{w \leq n} g_w = (\theta_{n+1} - \theta_1)/\eta$. Next we bound θ from above. Every send satisfies $D_c \geq \lambda p_i b$ with $D_c \leq \Delta_{\max}$, $p_i \geq 1$ and $b \geq b_{\min}$. If $\lambda > \lambda^\dagger = \Delta_{\max}/b_{\min}$, no send can happen, so $\text{spent}_w = 0$, $g_w = -1$, and θ decreases. Since one update raises θ by at most 2η , $\theta_w \leq \max(\theta_1, \log \lambda^\dagger) + 2\eta$ for all w . Substituting, $\frac{1}{n} \sum_{w \leq n} g_w \leq \frac{\log(\lambda^\dagger e^{2\eta}/\lambda_0)}{\eta n}$, using $\theta_1 = \log \lambda_0 \leq \log \lambda^\dagger$ w.l.o.g. The left-hand side is the average relative overspend, clipped at 2; overspends larger than $2B_i$ in a window are already excluded by the bucket whenever $\kappa_i \leq 3B_i$. $\square$